



贪心算法

参考《算法导论》





Contents

- ◆ 1 活动安排问题
- ◆ 2 贪心算法的原理
- ◆ 3 哈夫曼编码
- ◆ 4 最小生成树
- ◆ 5 单源最短路



1 活动安排问题（《算法导论》）

- ◆ n 个活动申请一个活动室, 各活动起始终止区间 (s_i, f_i)
- ◆ 输入: $n, (s_i, f_i), i \in [1, n]$
- ◆ 输出: 最大相容活动子集(活动之间无冲突, **活动个数最多**)
- ◆ 例:
 - ☞ 相容活动子集: $\{a_3, a_9, a_{11}\}$
 - ☞ 最大相容活动子集: $\{a_1, a_4, a_8, a_{11}\}$

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

按终止时间排序 $f_1 \leq f_2 \leq \dots \leq f_n$.

1 活动安排问题-贪心算法

- ◆ 贪心选择：选择一个可以加入到最优解中的活动
- ◆ 贪心选择：**选择最早结束的活动**
- ◆ 算法过程：
 1. 按终止时间排序
 2. 选择最早结束的活动，即第一个活动
 3. 在其余活动集合中选择与已选活动不冲突且结束时间最早的活动。

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

按终止时间排序 $f_1 \leq f_2 \leq \dots \leq f_n$.

1 活动安排问题-贪心算法

```
Greedy-Activity-Selector (int * s, int * f, int n)
{//假设活动已经按照结束时间排序
    A = {a1} ; //选择最早结束的活动a1。
    k = 1; //设k为下一选择活动的前驱活动
    for( m = 2; m <= n; m++) { //按结束时间依次考察各个活动
        if( s[m] >= f[k] ) { // m开始时间大于k结束时间
            A = A ∪ {am}; //选择下一个活动m
            k = m; //更新前驱
        } //if
    } //for
    return A;
} // Greedy-Activity-Selector
```

算法正确性证明

◆ 定义：任务集合 S_k ： a_k 结束后开始的任務集合，即

$$S_k = \{a_i \in S : s_i \geq f_k\}$$

◆ 例如：

¶ $S_1 = \{a_4, a_6, a_7, a_8, a_9, a_{11}\}$

¶ $S_2 = \{a_4, a_6, a_7, a_8, a_9, a_{11}\}$

¶ $S_3 = \{a_7, a_8, a_9, a_{11}\}$

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

算法正确性证明

- ◆ 证明1：活动选择问题具有最优子结构性质
- ◆ (最优子结构性质：问题的最优解包含其子问题的最优解)
- ◆ 证明：
 - ✎ 假设A是包含 a_1 的最大相容活动集(最优策略)
 - ✎ 因为 a_1 是最早结束的活动,
 - ✎ 所以与 a_1 兼容的活动都必须在 a_1 结束之后开始。
 - ✎ 即 $A - \{a_1\}$ 是剩下的 S_1 (子问题)上的最大相容活动集

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

算法正确性证明

◆ **证明2：贪心选择是最优解的一部分。**

◆ **定理16.1** 对于任意非空子问题 S_k ，令 a_m 是 S_k 中结束最早的活动，则 a_m 在 S_k 的某个最大相容子集中。

◆ **证明：**

¶ 令 A_k 是 S_k 的一个最大相容子集，且 a_j 是 A_k 中结束最早活动

¶ 若 $a_j = a_m$ ，则得证。

¶ 若 $a_j \neq a_m$ ，则 $f_j \geq f_m$ 。将 A_k 中的 a_j 替换为 a_m ，得到子集 A'_k

¶ $|A'_k| = |A_k|$

¶ A'_k 是 S_k 的一个最大相容子集。

$\{a_2, a_4, a_8, a_{11}\}$

$\{a_1, a_4, a_8, a_{11}\}$

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16



◆ 第一步 做出贪心选择：

- 选择一个活动，加入到最优解中。
即选择最早结束的活动 a_1 。

◆ 第二步 求解剩余子问题:

- 即寻找 a_1 结束后开始的活动。
- 若 A 是包含 a_1 的最大相容活动集(最优策略),
则 $A - \{a_1\}$ 是剩下的 S_1 (子问题)上的最大相容活动集

◆ **证明算法的正确性：**即证明贪心选择总是最优解的一部分

- ## 数学归纳法



2 贪心算法原理

◆ 贪心算法的设计步骤

- I. 将最优化问题转化为下面的形式：**做出一次选择后，只剩下一个子问题需要求解**
- II. **证明算法正确性：**
 - a) **贪心选择性：**即可以通过做出局部最优选择来构造全局最优解。
 - b) **最优子结构性：**一个问题的最优解包含其子问题的最优解。

即做出贪心选择后，剩余子问题的最优解与贪心选择组合即可得到原问题的最优解。



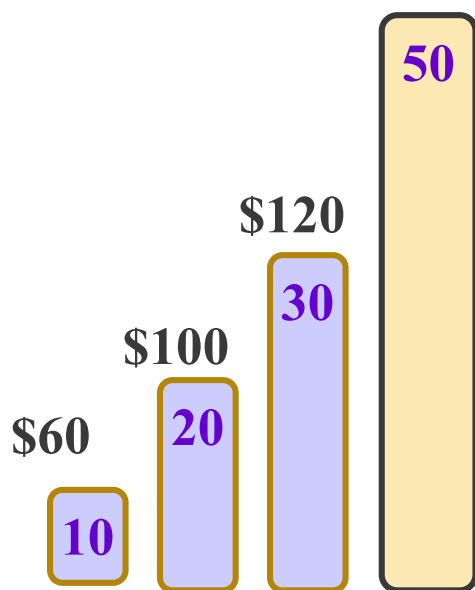
2 贪心算法原理

- ◆ 贪心对动态规划

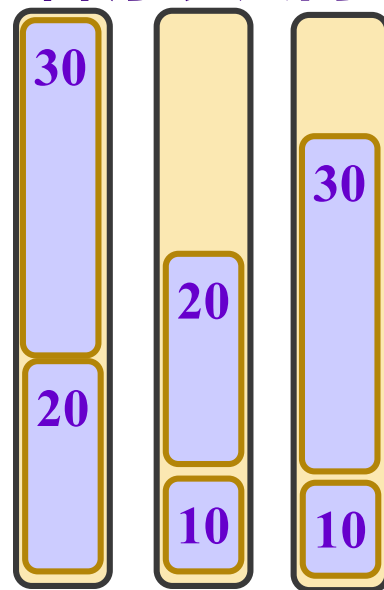
- ◆ 是否满足贪心选择性

- ┆ 0-1背包问题与分数背包问题

- ┆ 分数背包问题满足：按照每单位价值降序装入商品即可



分数背包问题满足



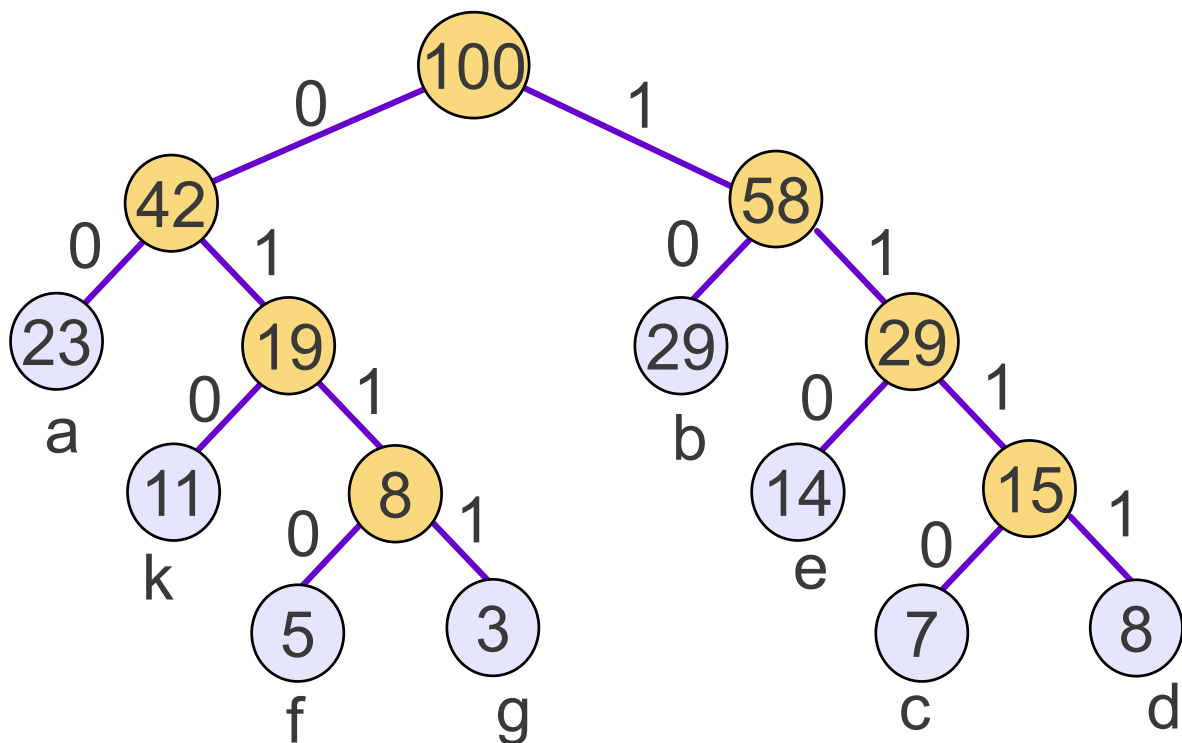
0-1背包问题不满足



3 哈夫曼树

例：某通讯系统只使用8种字符，使用哈夫曼树设计不等长编码。已知字符和对应使用频率如下：

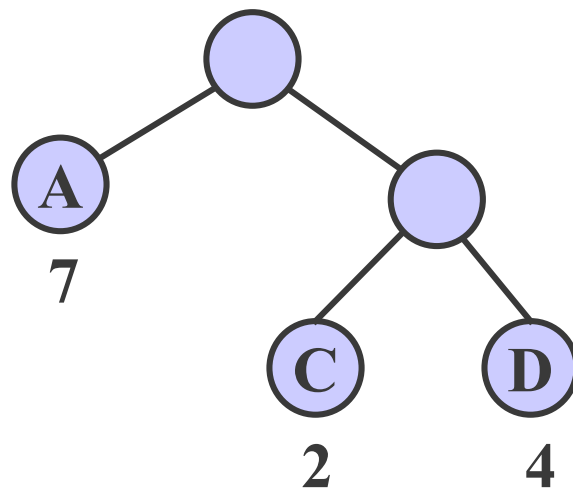
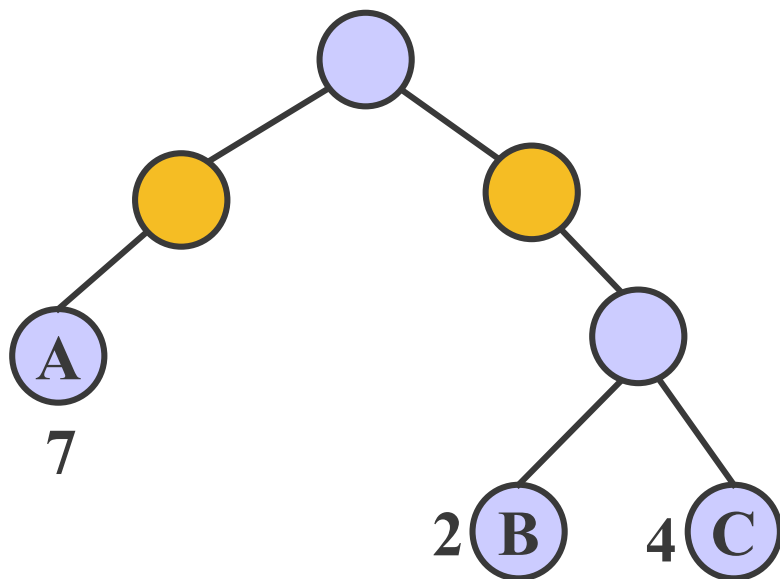
字符	a	b	c	d	e	f	g	k
频率%	23	29	7	8	14	5	3	11



a: 00
b: 10
c: 1110
d: 1111
e: 110
f: 0110
g: 0111
k: 010

哈夫曼算法的正确性(《算法导论》)

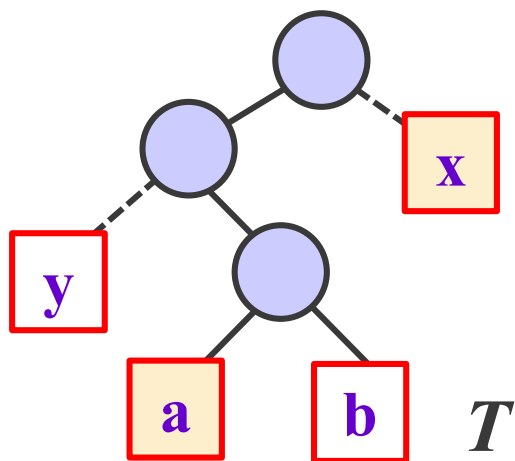
- ◆ 0) 证明哈夫曼树一定是正则的二叉树
- ◆ 1) 证明哈夫曼树问题具有贪心选择性质
- ◆ 2) 证明哈夫曼树问题具有最优子结构性质



哈夫曼树一定是正则的二叉树

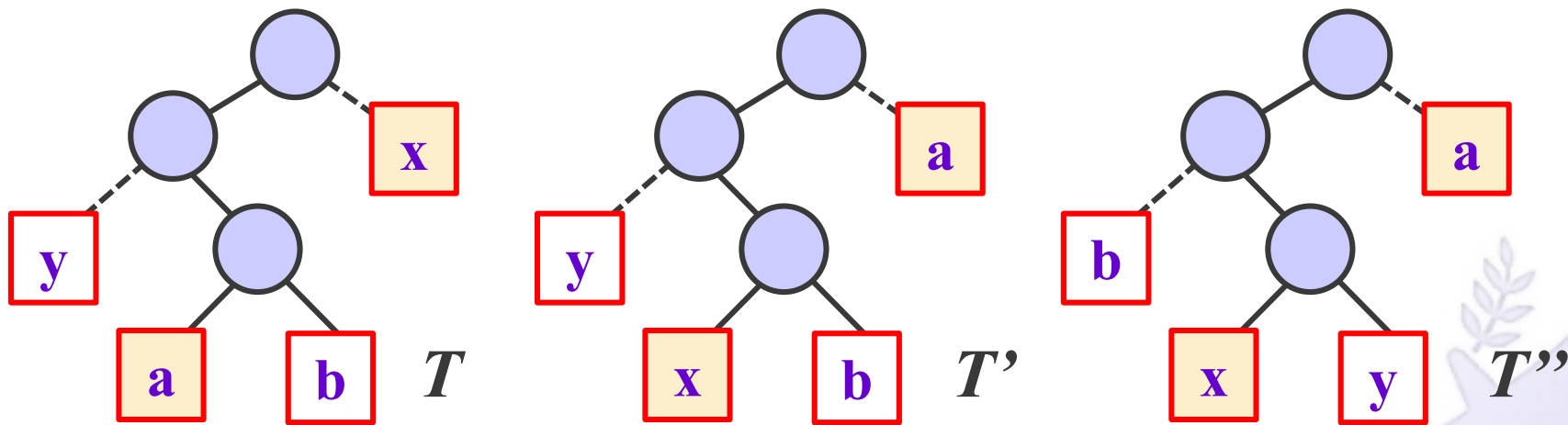
哈夫曼算法的正确性

- ◆ 1) 证明哈夫曼编码问题具有**贪心选择性质**
- ◆ **引理16.2**: 令 C 为一个字母表, 其中每个字符 c 的频率为 f_c 。令 x 和 y 是 C 中频率最低的两个字符, 设 $f_x \leq f_y$ 。那么**存在 C 的一棵最优树, 其中 x 和 y 互为兄弟节点。**
- ◆ **证明**: 令 T 是任意一个哈夫曼树, 令 a 和 b 是 T 中深度最大的兄弟叶节点, 设 $f_a \leq f_b$ 。则有 $f_x \leq f_a$ 且 $f_y \leq f_b$ 。



哈夫曼算法的正确性

- ◆ T 中交换节点 a 和 x 得到 T' , T' 中交换节点 b 和 y 得到 T'' .
- ◆ 则树 T 和 T' 的代价满足: $WPL(T) \geq WPL(T')$;
- ◆ 树 T 和 T'' 的代价满足: $WPL(T) \geq WPL(T'')$;
- ◆ 又由于 T 是最优树, 所以 $WPL(T'') = WPL(T)$, 即 T'' 也是最优树。



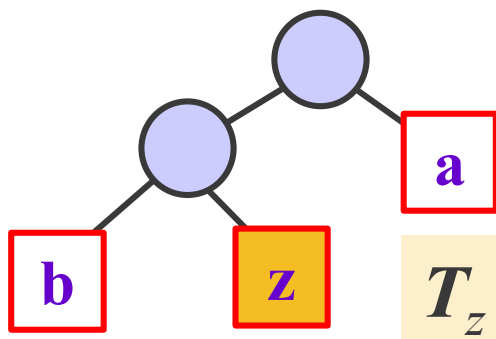
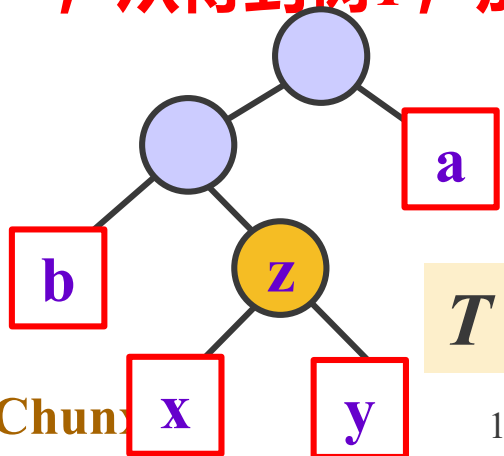
$$f_x \leq f_a \text{ 且 } f_y \leq f_b$$

哈夫曼算法的正确性

◆ 2) 证明哈夫曼树问题具有最优子结构性质

◆ 引理16.3:

- 令 C 为一个字母表，其中每个字符 c 的频率为 f_c 。
- 令 x 和 y 是 C 中频率最低的两个字符，设 $f_x \leq f_y$ 。
- 令 C_z 为去掉字符 x 和 y ，加入新字符 z 得到的 ($f_z = f_x + f_y$)。
- 令 T_z 为 C_z 的任意一个哈夫曼树。
- 则：将 T_z 中叶节点 z 替换为一个以 x 和 y 为子节点的内部节点，从得到树 T ，那么 T 是字母表 C 的一棵哈夫曼树。



$$WPL(T) = WPL(T_z) + f_x + f_y;$$

哈夫曼算法的正确性

$$WPL(T) = WPL(T_z) + f_x + f_y;$$

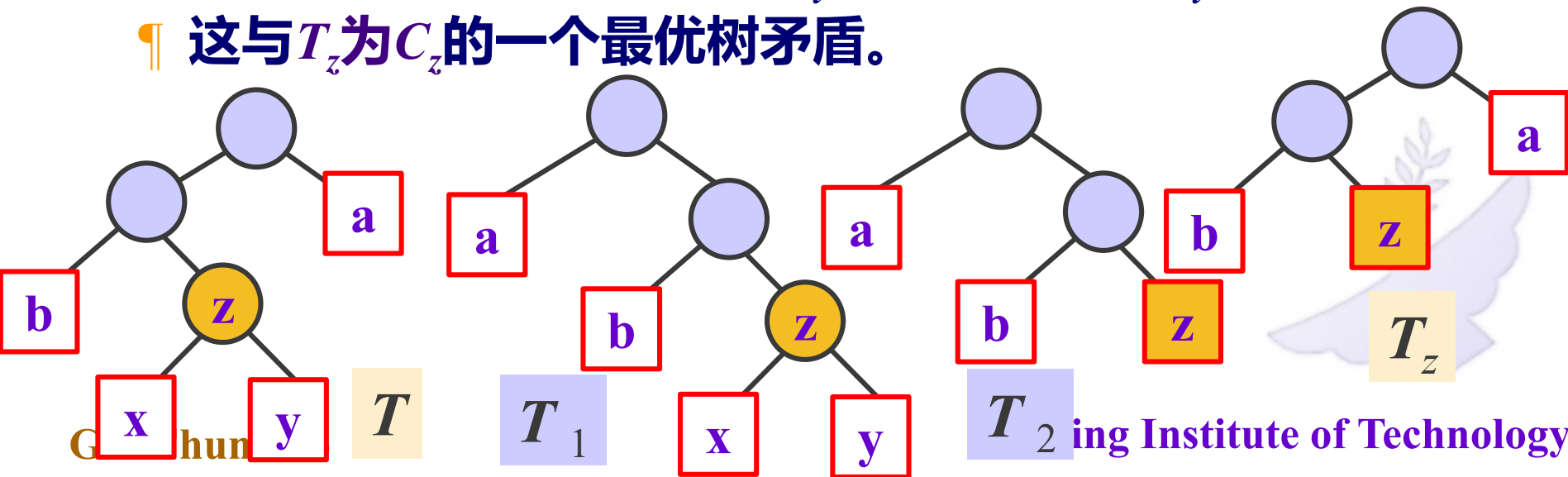
◆ 证明：反证法

1) 假设 T 不是 C 的哈夫曼树，**必存在**一棵最优树 T_1 ，根据**贪心选择性**， T_1 包括频率最小的节点 x 和 y ，且为兄弟节点则：
$$WPL(T_1) < WPL(T)$$

2) 令 T_2 为将 T_1 中 x 和 y 及其父节点替换为叶节点 z 得到的树，则：

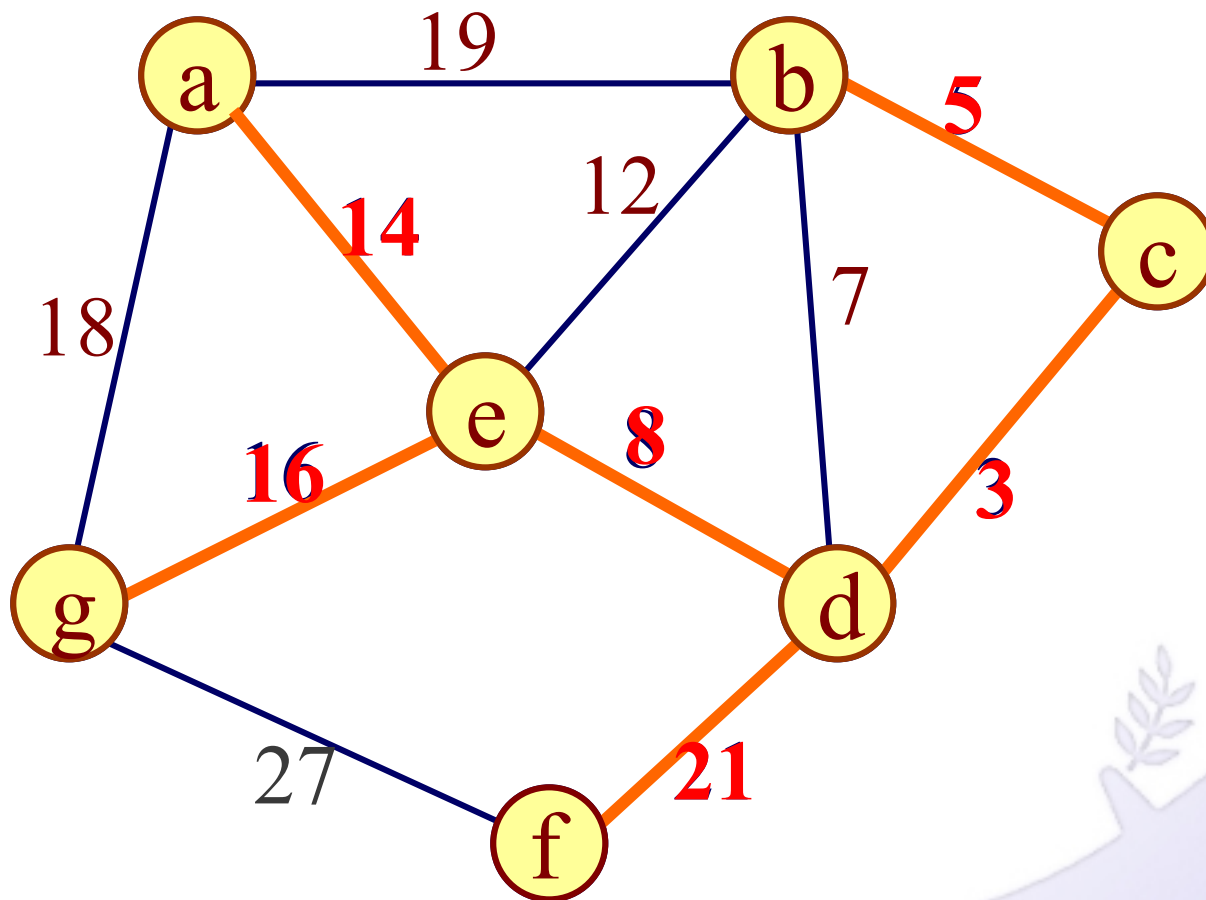
$$WPL(T_2) = WPL(T_1) - f_x - f_y < WPL(T) - f_x - f_y = WPL(T_z)$$

这与 T_z 为 C_z 的一个最优树矛盾。



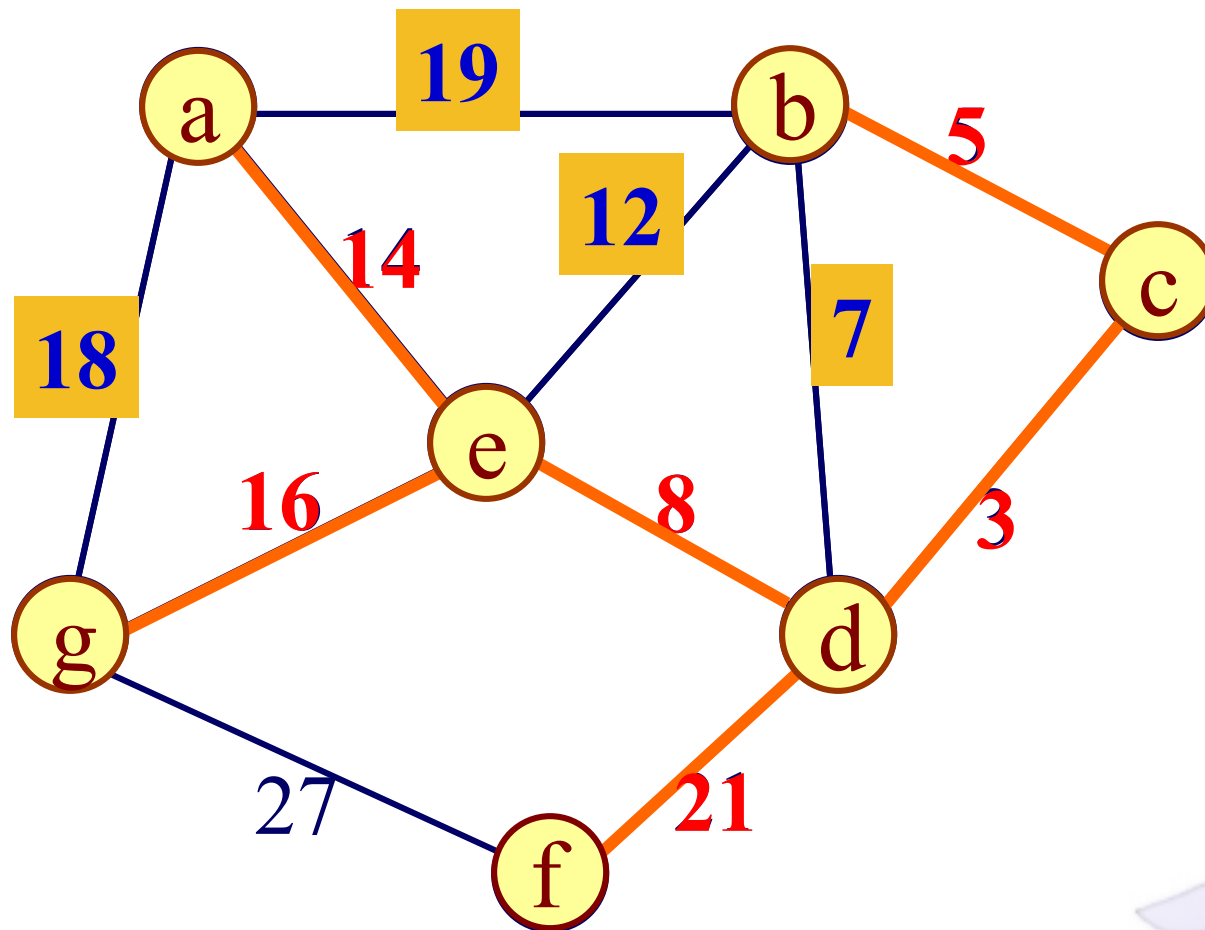
4 最小生成树

◆ Prim



4 最小生成树

◆ Kruskal





4 最小生成树

- ◆ 1) 证明最小生成树问题 具有贪心选择性质
 - ┌ 即证明贪心选择的边属于某棵最小生成树
- ◆ 2) 证明最小生成树问题 具有最优子结构性质
 - ┌ (最优子结构性质: 问题的最优解包含其子问题的最优解)

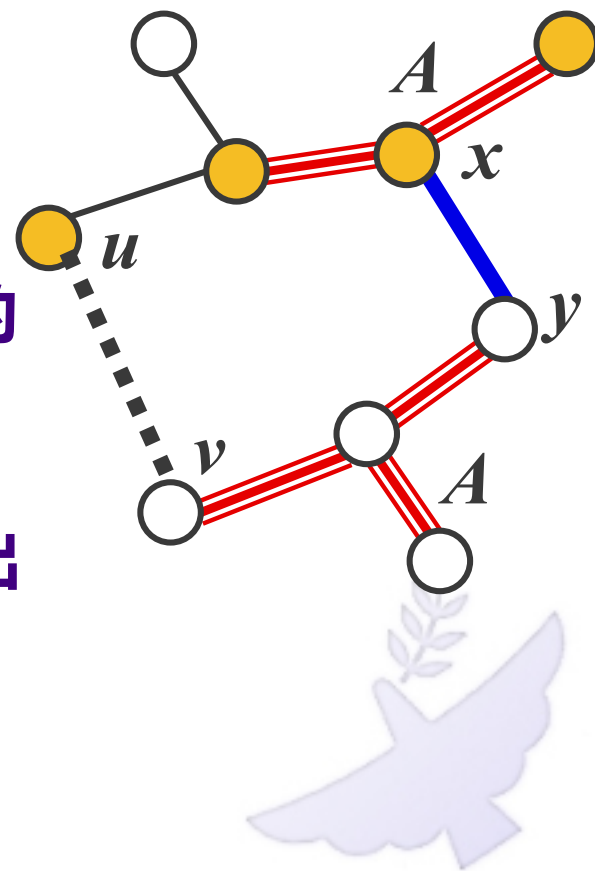


4 最小生成树

- ◆ 1) 证明具有贪心选择性质
- ◆ 即证明贪心选择的边属于某棵最小生成树
- ◆ 定理23.1

- ✎ 设 $G=(V, E)$ 是一个连通无向图，边上定义了实数权重函数
- ✎ 设集合 A 是 E 的一个子集，且包括在 G 的某棵最小生成树 T 中。
- ✎ 设 $(S, V-S)$ 是 G 的一个切割，且 A 的边要么在 S 的导出子图中，要么在 $V-S$ 的导出子图中。
- ✎ 设 (u, v) 是横跨切割 $(S, V-S)$ 的一条权重最小的边。

- ◆ 那么 (u, v) 包含在 G 的某棵最小生成树中



4 最小生成树

◆ 证明:

- 设 T 是包含 A 的最小生成树
- 若 T 包含边 (u, v) , 则得证
- 若 T 不包含边 (u, v) , 则 T 中必定包含一条边连通 S 和 $V-S$, 设为 (x, y) . 显然

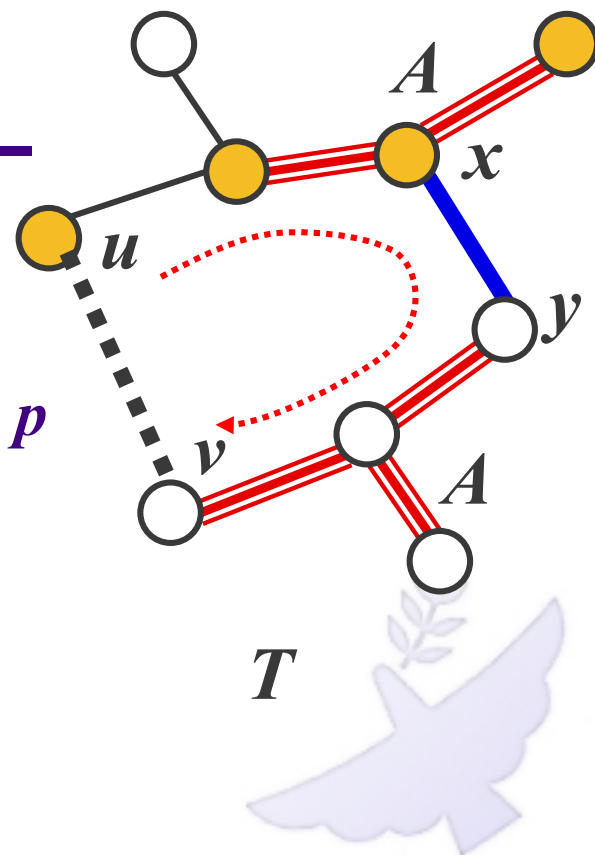
$$w(u, v) \leq w(x, y)$$

- 则 T 中必定存在一条从 u 到 v 的路径 p , p 包含边 (x, y) 。

- 令树 $T' = T - \{(x, y)\} \cup \{(u, v)\}$, 则

$$w(T') \leq w(T)$$

◆ 所以 T' 也是 G 的最小生成树



4 最小生成树

◆ 2) 证明具有最优子结构性质

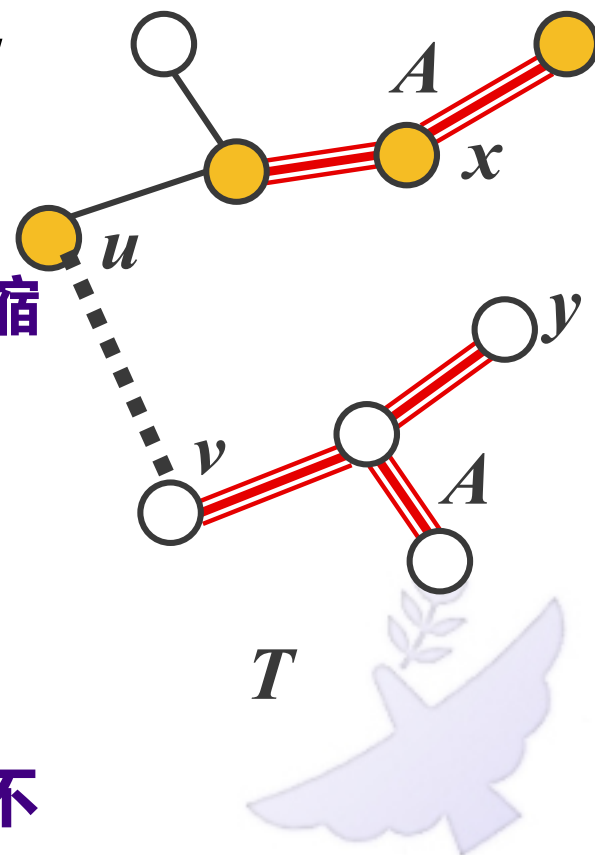
◆ (最优子结构性质: 问题的最优解包含其子问题的最优解)

◆ 证明: 已知 A 属于某棵最小生成树 T , $A \subseteq T$, 由上述证明1) 知: $A \cup \{(u, v)\}$ 属于某一棵最小生成树, $A \cup \{(u, v)\} \subseteq T'$

✎ 将 $A \cup \{(u, v)\}$ 中的各个连通分支进行收缩, 每个分支收缩为一个点, 得到图 G' . 设 G' 最小生成树为 T'' , 则

✎ $A \cup \{(u, v)\} \cup T''$ 即为 G 的最小生成树。

✎ 显然构造 T'' 不需要考虑 $A \cup \{(u, v)\}$ 中的边(已选边)和收缩过程中收缩的其它边(内边), 即子问题 G' 与 $A \cup \{(u, v)\}$ 不相交。





单源最短路径问题 (Dijkstra算法)

- ◆ 1) 证明单源最短路径问题 具有**贪心选择性质**
 - 🔧 证明: 每次选择的点即得到起点到该点的最短路径
- ◆ 2) 证明单源最短路径问题 具有**最优子结构性质**
 - 🔧 证明: 最短路径的子路径也是最短路径





-

回顾：Dijkstra算法

◆ 辅助集合S:

- 🔑 当前已经得到最短路径的顶点集合
- 🔑 初始时, $S=\{V_0\}$

◆ 辅助数组Dist

- 🔑 求解过程中, $\text{Dist}[k]$ 表示 “当前” 所求得从源点到顶点 k 的最短路径
- 🔑 $\text{Dist}[k] = \langle \text{源点到顶点 } k \text{ 的弧上的权值} \rangle$
- 🔑 或者 $\text{Dist}[k] = \langle \text{源点到顶点 } j \text{ 的路径长度} \rangle + \langle \text{顶点 } j \text{ 到顶点 } k \text{ 的弧上的权值} \rangle$



单源最短路径问题 (Dijkstra算法)

◆ 1) 证明具有贪心选择性质

◆ **即证明：**从 $\text{Dist}[]$ 中选择具有最小值的节点 u 时就得到从 s 到的 u 一条最短路径，即 $d[u] = \delta(s, u)$.

◆ **证明：** (数学归纳法)

‣ 归纳基础: 初始 $S = \{s\}$, $d[s] = \delta(s, s) = 0$, 性质成立.

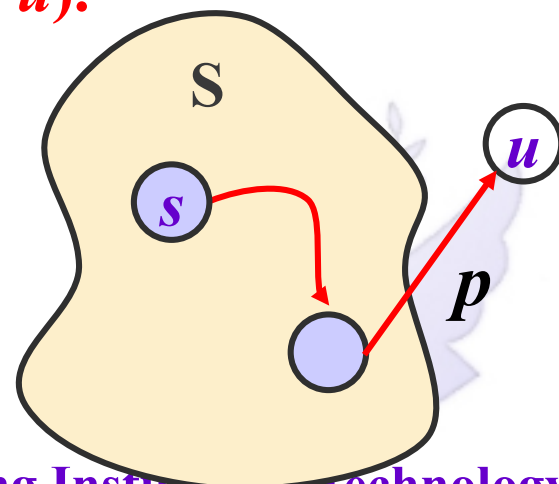
‣ 归纳假设: 在某一步性质成立, 即 $\forall v \in S, d[v] = \delta(s, v)$.

‣ 归纳证明: 从 $\text{Dist}[]$ 中取出 u 时 $d[u] = \delta(s, u)$.

◆ **反证法：**

‣ 取出 u 时得到一条路径 p , 长度为 $d[u]$

‣ **假设 p 不是最短路径, 那么 $d[u] > \delta(s, u)$.**





-
- The top diagram shows a yellow irregular shape labeled S . Inside, there is a node s (purple circle with s). A red arrow starts at s , goes down and right to another purple circle, then continues as a straight red arrow to a node u (purple circle with u) outside the shape. The label p is at the bottom right.
- The bottom diagram shows a yellow irregular shape labeled S . Inside, there is a node s (purple circle with s). A red arrow starts at s , goes down and right to a node x (purple circle with x), then continues to a node y (purple circle with y) outside the shape. Another red arrow starts at s , goes up and right to a node u (purple circle with u) outside the shape. The label γ is at the bottom right.



贪心算法原理

◆ 贪心算法的设计步骤

- I. 将最优化问题转化为下面的形式：**做出一次选择后，只剩下一个子问题需要求解**
- II. **证明算法正确性：**
 - a) **贪心选择性：**即可以通过做出局部最优选择来构造全局最优解。
 - b) **证明最优子结构性：**一个问题的最优解包含其子问题的最优解。

即做出贪心选择后，剩余子问题的最优解与贪心选择组合即可得到原问题的最优解。





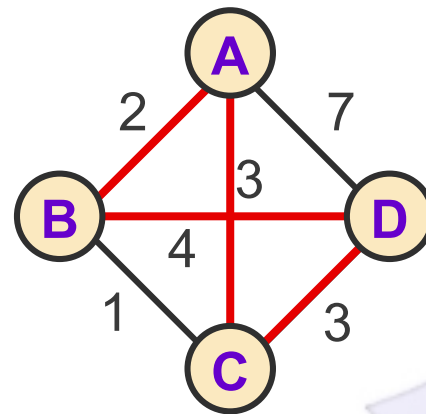
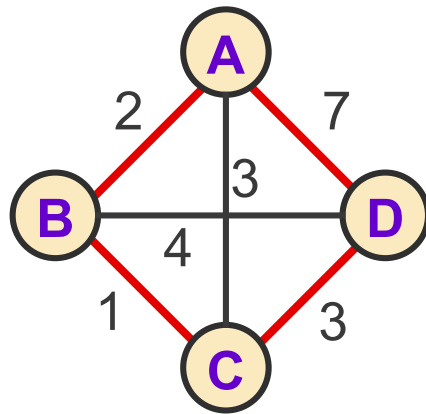
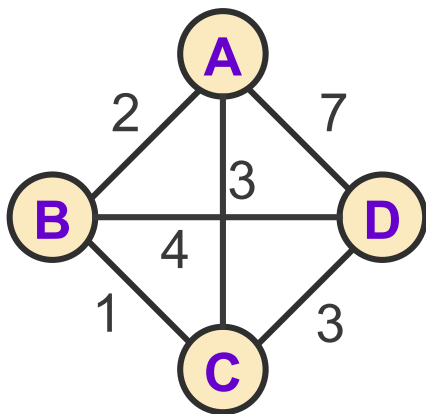
构造“贪心”反例

- ◆ **找零问题：**一出纳员支付一定数量的现金。假设他手中有各种面值的纸币和硬币，要求他用最少的货币数支付规定的现金。
- ◆
- ◆ 例如，现有4种硬币：它们的面值分别为1分、2分、5分和1角，要支付2角5分
- ◆ 首先支付2个1角硬币，然后支付一个5分硬币，这就是贪心策略
- ◆
- ◆ 反例：现有3种硬币：它们的面值分别为1分、4分、和6分，要支付8分



构造“贪心”反例

- ◆ **货郎担 (TSP) 问题:** 设售货员要到五个城市去售货, 最后再回到出发的城市, 已知从一个城市到其他城市的费用, 求总费用最少的路线.
- ◆ **贪心策略:** 采用类似MST的策略
 - BC, BA, CD, AD: 13
- ◆ **最优解:** 12





- ◆ 1. 字符a~h出现的频率恰好是前8个Fibonacci数,
 - 🔧 它们的Huffman编码是什么?
 - 🔧 将结果推广到n个字符的频率恰好是前n个Fibonacci数的情形.
- ◆ 2. 若在0-1背包问题中, 各物品依重量递增排列时, 其价值恰好降序排列, 对这个特殊的0-1背包题, 设计一个有效算法找出最优解, 并说明算法的正确性.





作业题

- ◆ 3. 最优分解问题.
- ◆ **问题描述:** 设 n 是一个正整数, 将 n 分解为若干互不相同的自然数之和, 且使这些自然数的乘积最大.
- ◆ **算法设计:** 对于给定的正整数 n , 计算最优分解方案.
- ◆ **数据输入:** 正整数 n .
- ◆ **结果输出:** 将计算的最大乘积输出到文件output.txt
- ◆ 例如若 $n=10$, 则最优分解为 $2+3+5$, 最大乘积为30.





分发饼干

- ◆ **问题描述：**假设要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。对每个孩子 i ，都有一个胃口值 g_i ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 s_j 。如果 $s_j \geq g_i$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。
- ◆ **解题思路，**首先需要将胃口值和饼干尺寸由小至大排序。设定一个计数器 $child$ ，用来记得到满足的孩子个数，再维护一个饼干指针 $cookies$ 。如果饼干尺寸可以满足孩子胃口值，即 $g[child] \leq s[cookies]$ ，就将 $child$ 、 $cookies$ 分别加一(向后移动一位)，否则只将 $cookies$ 向后移动一位。因为孩子的胃口值是由小到大的，若不满足当前的胃口值更不会满足之后的。



55. 跳跃游戏

- ◆ **题目描述：** 给定一个非负整数数组，你最初位于数组的第一个位置。数组中的每个元素代表你在该位置可以跳跃的最大长度。判断你是否能够到达最后一个位置。





435.无重叠区间

- ◆ 题目描述：给定一个区间的集合，找到需要移除区间的最小数量，使剩余区间互不重叠。
- ◆ 注意：
- ◆ 可以认为区间的终点总是大于它的起点。
- ◆ 区间 $[1,2]$ 和 $[2,3]$ 的边界相互“接触”，但没有相互重叠。





376. 摆动序列

- ◆ 如果相邻数字之间的差严格地在正数和负数之间交替，则数字序列称为摆动序列。第一个差（如果存在的话）可能是正数或负数。少于两个元素的序列也是摆动序列。
- ◆ 例如， $[1,7,4,9,2,5]$ 是一个摆动序列，因为差值 $(6,-3,5,-7,3)$ 是正负交替出现的。相反， $[1,4,7,2,5]$ 和 $[1,7,8,6,4,2,3]$ 不是摆动序列。
- ◆ 给定一个整数序列，返回作为摆动序列的最长子序列的长度。通过从原始序列中删除一些（也可以不删除）元素来获得子序列，剩下的元素保持其原始顺序。
- ◆ 输入： $[1,7,4,9,2,5]$ ，输出 6
- ◆ 输入： $[1,4,7,2,5]$ ，输出 4
- ◆ 输入： $[1,7,8,6,4,2,3]$ ，输出 4





餐馆问题

- ◆ 题目：某餐馆有 n 张桌子，每张桌子能容纳的最大人数为 a ；有 m 批客人，每批客人有两个参数： b 人数， c 预计消费金额数。在不允许拼桌的情况下，请选择其中一部分客人，使得总预计的消费金额最大。
- ◆ 输入：总共输入 $m+2$ 行
- ◆ 第一行输入 $n\ m$,即桌子的个数和客人的批数（ $1 \leq n \leq 50000; 1 \leq m \leq 50000$ ）
- ◆ 第二行是 n 个 a ,即每张桌子能容纳的客人数
- ◆ 接下来 m 行输入： $b\ c$,即每批客人的人数和预计消费金额
- ◆ 输出：一个整数，即总预计消费金额





- ◆ 测试实例： 3 5
- ◆ 2 4 2
- ◆ 1 3
- ◆ 3 5
- ◆ 3 7
- ◆ 5 9
- ◆ 1 10
- ◆ 输出： 20





- ◆ 思路：先用贪心算法对每批客人的消费金额进行降序排序，对金额相同的客人的人数进行升序排序；然后用二分法枚举每批客人去最合适的桌子。

